

PHY1133: Python for Physicists
Academic Year: 2025–2026
Planning a Flight Route over the Mediterranean
Dr Andrea De Marco

Submission Deadline: [31 January 2026]

1 Scenario

You are helping a small cargo aircraft plan a route to deliver equipment to several airports around the Mediterranean. The pilot wants a route that

- starts at a home base (Malta International Airport),
- visits every other airport exactly once,
- returns to Malta, and
- has the shortest possible distance along Earth’s surface.

This is an instance of the Traveling Salesperson Problem (TSP) on a sphere. In this assignment you will:

- work with real-looking latitude/longitude coordinates,
- implement the Haversine formula for distances on Earth,
- implement two TSP algorithms:
 1. brute force search (exact, small problems),
 2. nearest neighbour heuristic (fast, approximate),
- compute and compare route lengths, and
- visualise the routes.

You should use Python tools you already know: variables, conditions, loops, lists, functions, basic NumPy, and simple plotting.

2 Data: Airports Around the Mediterranean

You are given 10 airports with approximate latitude and longitude in degrees:

```
# name, latitude (deg), longitude (deg)
airports = [
    ("Malta Int. (MLA)",      35.857,  14.477),
    ("Catania (CTA)",        37.469,  15.066),
    ("Palermo (PMO)",        38.176,  13.103),
    ("Rome Fiumicino (FCO)", 41.800,  12.238),
    ("Athens (ATH)",         37.936,  23.947),
    ("Naples (NAP)",         40.886,  14.290),
    ("Tunis (TUN)",          36.851,  10.228),
    ("Algiers (ALG)",        36.691,   3.215),
    ("Barcelona (BCN)",      41.297,   2.078),
    ("Nice (NCE)",           43.665,   7.215),
]
```

Treat index 0 ("Malta Int. (MLA)") as the fixed starting and ending airport. Tours will start at index 0, visit all indices 1–9 exactly once, then return to 0.

3 Coordinates, Distances, and Physics

3.1 Latitude and Longitude

- Latitude (lat): north–south angle from the equator.
 - 0° at the equator,
 - $+90^\circ$ at the North Pole,
 - -90° at the South Pole.
- Longitude (lon): east–west angle from Greenwich.
 - 0° at Greenwich,
 - positive eastwards, negative westwards.

Python’s trigonometric functions (`math.sin`, `math.cos`, etc.) use radians, not degrees. The conversion from degrees to radians is

$$\text{radians} = \text{degrees} \times \frac{\pi}{180}.$$

3.2 Flat Euclidean Distance

If we pretend the Earth is flat over this small region and use latitude and longitude as y and x , we can define a local planar distance in *kilometres* by converting degrees to km first. Let $P = (\text{lat}_1, \text{lon}_1)$ and $Q = (\text{lat}_2, \text{lon}_2)$ in degrees and define

$$\Delta\text{lat} = \text{lat}_2 - \text{lat}_1, \quad \Delta\text{lon} = \text{lon}_2 - \text{lon}_1.$$

Using Earth’s radius $R = 6371$ km, set

$$k_{\text{lat}} = \frac{\pi R}{180}, \quad k_{\text{lon}} = k_{\text{lat}} \cos(\bar{\varphi}), \quad \bar{\varphi} = \frac{\text{lat}_1 + \text{lat}_2}{2} \cdot \frac{\pi}{180} \text{ (radians)}.$$

The local Euclidean distance in km is then

$$d_{\text{euclid}} = \sqrt{(k_{\text{lon}} \Delta\text{lon})^2 + (k_{\text{lat}} \Delta\text{lat})^2}.$$

This “flat” approximation is not physically exact over large areas, but on this dataset it is acceptable for comparison with Haversine.

3.3 Haversine Distance (Great-Circle Distance)

To approximate the physical distance along Earth’s surface, we use the Haversine formula.

For two points with coordinates $(\text{lat}_1, \text{lon}_1)$ and $(\text{lat}_2, \text{lon}_2)$ in degrees, we define

$$\begin{aligned} \Delta\text{lat} &= \text{lat}_2 - \text{lat}_1, \\ \Delta\text{lon} &= \text{lon}_2 - \text{lon}_1. \end{aligned}$$

After converting all angles to radians, the Haversine expression is

$$a = \sin^2\left(\frac{\Delta\text{lat}}{2}\right) + \cos(\text{lat}_1) \cos(\text{lat}_2) \sin^2\left(\frac{\Delta\text{lon}}{2}\right).$$

The central angle c is

$$c = 2 \cdot \arctan 2(\sqrt{a}, \sqrt{1-a}),$$

and the Haversine distance is

$$d_{\text{haversine}} = R \cdot c,$$

where R is Earth's radius in km (use $R = 6371$ km). You must implement this function.

4 The Traveling Salesperson Problem (TSP)

We have $N = 10$ airports, indexed from 0 to 9. A *tour* is a list of indices

$$[0, i_1, i_2, \dots, i_9, 0]$$

where 0 appears at the beginning and the end, and each of the other indices 1–9 appears exactly once. The length of a tour is the sum of distances between consecutive airports along the route, using the chosen distance metric (Haversine distance in km). The goal is to find a tour with the smallest possible total Haversine distance.

5 Algorithms to Implement

You must implement both of the following algorithms:

1. brute force search (exact),
2. nearest neighbour heuristic (constructive).

5.1 Brute Force Search

Strategy:

- Fix starting index 0 (Malta).
- Generate all permutations of the remaining indices $[1, 2, \dots, 9]$.
- For each permutation, build the route $[0] + \text{perm} + [0]$ and compute its length.
- Keep the shortest route and its length.

Pseudocode outline:

1. `start = 0`.
2. `others = [1, 2, ..., 9]`.
3. Set `best_route = None` and `best_length = +∞`.
4. For each permutation `perm` of `others`:
 - (a) Build `route = [start] + list(perm) + [start]`.
 - (b) Compute `L = route_length(route, airports, haversine_distance)`.
 - (c) If `L < best_length`, update the best route and length.
5. Return `best_route` and `best_length`.

You may use `itertools.permutations` to generate permutations.

5.2 Nearest Neighbour Heuristic

Strategy:

- Start at Malta.
- At each step, move to the closest unvisited airport.

- After visiting all airports, return to Malta.

Pseudocode:

1. `start = 0`.
2. `current = start`.
3. `route = [current]`.
4. `unvisited = {1, 2, ..., 9}`.
5. While `unvisited` is not empty:
 - (a) For each candidate in `unvisited`, compute the distance from `current`.
 - (b) Choose the candidate with the smallest distance.
 - (c) Append it to `route`, remove it from `unvisited`, and set `current = candidate`.
6. Append `start` to `route` to close the loop.
7. Return `route`.

You should then compute the length of this route using Haversine distance.

6 Programming Tasks

Submit a single `.py` file or a Jupyter notebook.

Task 1: Data and Helpers

1. Store the `airports` list as given.
2. Implement a helper to get coordinates by index.

Task 2: Distance Functions

1. Implement `deg2rad(deg)`.
2. Implement

```
def euclidean_distance(p, q, R=6371.0):
    ...
```

where `p` and `q` are `(lat_deg, lon_deg)`; return the local flat-plane distance *in kilometres* using the small-area conversion $k_{\text{lat}} = \pi R / 180$ and $k_{\text{lon}} = k_{\text{lat}} \cos(\bar{\varphi})$ with $\bar{\varphi}$ the mean latitude in radians (see the “Flat Euclidean Distance” subsection).

3. Implement

```
def haversine_distance(p, q, R=6371.0):
    ...
```

where `p` and `q` are `(lat_deg, lon_deg)`.

4. Implement a route length function

```
def route_length(route, airports, distance_fn=haversine_distance):
    """route is a list like [0, 3, 5, ..., 0]; returns total distance in km."""
```

looping over consecutive index pairs in `route`. Use the `distance_fn` argument so you can pass either `haversine_distance` or `euclidean_distance`.

5. Optional: compare route lengths using both metrics (Euclidean vs Haversine) and comment briefly.

Task 3: TSP Algorithms

1. Implement brute force:

```
def brute_force_route(airports, start_index=0):  
    """Returns (best_route, best_length) using exhaustive search."""
```

2. Implement nearest neighbour:

```
def nearest_neighbour_route(airports, start_index=0):  
    """Returns a route using the nearest neighbour heuristic."""
```

3. Run both algorithms:

- brute force to get an exact best route (`route_exact`, `L_exact`),
- nearest neighbour to get a heuristic route (`route_nn`, `L_nn`).

Print and compare both route lengths.

7 Visualisation

Using `matplotlib`:

1. Plot all airports as points with
 - longitude on the x -axis,
 - latitude on the y -axis,
 - labels for each point with its name.
2. Plot two routes:
 - the exact route (brute force),
 - the nearest neighbour route.

You may either use different line styles/colours on the same figure or separate figures.

8 Short Written Answers

Answer briefly (a few sentences each) in your write-up:

1. In your own words, what is the Traveling Salesperson Problem?
2. Why is the Haversine formula more appropriate than a simple flat Euclidean distance for these coordinates?
3. Compare the two algorithms (brute force and nearest neighbour) in terms of
 - quality of the solution on this dataset (which route is shortest), and
 - what you expect to happen as the number of airports becomes very large.
4. If you experimented with another distance function, did the “best” route change?

9 Marking Scheme and Deliverables

The assignment also includes a write-up, which should be about 3 to 4 pages long, including figures and plots, but excluding any code listing. The write-up should include:

- A discussion of the problem and how you tackled it.

- An explanation of how the various aspects of your code works.
- Plots of the various route solutions.

Submit your source code, as a Python file or as a Colab Notebook along with your report in a ZIP file on the VLE.

The marking breakdown is as follows:

- Data representation and appropriate functions with separation of concerns: 15%.
- Correct distance function implementation and route length: 20%.
- Brute force implementation and result: 15%.
- Nearest neighbour implementation and result: 15%.
- Visualisation and clarity of plots: 15%.
- Written report with answers and interpretation: 20%.